G2_Project1: xv6 Customized System Calls Documentation

This documentation summarizes the 6 custom system calls implemented by our team in xv6. Below are detailed descriptions mapping exactly how the kernel behaves, accompanied by core C code snippet screenshots from our kernel patches, and terminal execution logs demonstrating correct functionality.

1. sem_create(), sem_down(), sem_up() System Calls
Implemented by: Abhinav Kumar

Objective:

The primary objective of this work is to implement process synchronization using counting semaphores in the xv6 operating system. User processes in xv6 are isolated from one another. This new functionality provides a mechanism for isolated processes to coordinate their execution. Specifically, this implementation enables a synchronization scenario where a parent process must securely wait for a signal from its child process before continuing.

Kernel Modificationsw

(i) System Call Definition

- **Added `SYS_sem_create`, `SYS_sem_down`, `SYS_sem_up` to `kernel/syscall.h`.** Unique system call numbers are assigned (e.g., 23, 24, 25) to uniquely identify these three operations.
- **Declared prototypes in `kernel/syscall.c`.** The kernel router is informed of the new system calls with `extern uint64 sys_sem_...()`.
- **Updated the system call table in `kernel/syscall.c`.** The specific numbers (e.g., `[SYS_sem_down] sys_sem_down`) are mapped to their respective function pointers.
- **Updated user-space generation in `user/usys.pl`.** Entries for `entry("sem_create");`, `entry("sem_down");`, and `entry("sem_up");` were added to generate the necessary RISC-V `ecall` assembly stubs.
- **Exposed prototypes to user space in `user/user.h`.** Prototypes like `int sem_create(int);` were added so user programs can call these functions with proper type checking.

(ii) Semaphore Data Structure and Storage

- **Defined `struct semaphore` in `kernel/sysproc.c`.** This structure holds essential synchronization information: a `spinlock` for race condition protection, an integer `value` for the count, and an `active` flag to manage allocation.

- **Allocated global semaphore storage in `kernel/sysproc.c`.** A fixed-size array `struct semaphore semaphores[NSEM];` was allocated within kernel memory. As the kernel is the ultimate authority, this array acts as the shared state accessible by all processes through system calls.

(iii) System Call Implementation

- **`sys_sem_create()`:** This system call retrieves an initial value from user space, finds an inactive semaphore slot in the kernel array, initializes its lock, sets the count, marks it active, and returns the unique semaphore index (ID) to the user.
- **`sys_sem_down()` (Wait/P):** This atomic operation decrements the semaphore count. If the count is currently 0, the calling process is made to block using a `while` loop that calls the kernel's internal `sleep()` function. Crucially, `sleep()` releases the semaphore's spinlock while the process blocks and atomically re-acquires it upon wake-up.
- **`sys_sem_up()` (Signal/V):** This atomic operation increments the semaphore count. It then calls the kernel's internal `wakeup()` function, which locates any processes sleeping on this specific semaphore's memory address and changes their state back to `RUNNABLE`.

Core Kernel Implementation (Screenshot Description)

```c
struct semaphore {
  struct spinlock lock; // Protects the semaphore from race conditions
  int value;            // The current count
  int active;           // 1 if allocated, 0 if free
} semaphores[NSEM];
```

```c
uint64
sys_sem_create(void)
{
  int initial_val;


  argint(0, &initial_val);

  if(initial_val < 0)
    return -1;

  for(int i = 0; i < NSEM; i++) {
    if(semaphores[i].active == 0) {
      initlock(&semaphores[i].lock, "semaphore");
      semaphores[i].value = initial_val;
      semaphores[i].active = 1;
      return i;
    }
  }
  return -1;
}
```

```c
uint64
sys_sem_down(void)
{
  int id;

  argint(0, &id);

  if(id < 0 || id >= NSEM || semaphores[id].active == 0)
    return -1;

  acquire(&semaphores[id].lock);

  while(semaphores[id].value == 0) {
    sleep(&semaphores[id], &semaphores[id].lock);
  }

  semaphores[id].value--;

  release(&semaphores[id].lock);
  return 0;
}
```

```c
uint64
sys_sem_down(void)
{
  int id;

  argint(0, &id);

  if(id < 0 || id >= NSEM || semaphores[id].active == 0)
    return -1;

  acquire(&semaphores[id].lock);

  while(semaphores[id].value == 0) {
    sleep(&semaphores[id], &semaphores[id].lock);
  }

  semaphores[id].value--;

  release(&semaphores[id].lock);
  return 0;
}
```

```
$ semtest
Starting Semaphore Test...
Parent: Waiting for child to finCish.
hild: I am doing some heavy work...
Child: Work finished. Sending signal to parent!
Parent: I received the signal. Child is done. Exiting.
$
```

```c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int main(int argc, char *argv[]) {
  // Create a semaphore initialized to 0
  int sem_id = sem_create(0);
  if(sem_id < 0) {
    printf("Failed to create semaphore\n");
    exit(1);
  }

  printf("Starting Semaphore Test...\n");
  int pid = fork();

  if (pid == 0) {
    // --- CHILD PROCESS ---

    printf("Child: I am doing some heavy work...\n");
    // Simulate time passing by keeping the CPU busy
    for (volatile int i = 0; i < 10000000; i++);
    printf("Child: Work finished. Sending signal to parent!\n");

    sem_up(sem_id); // Increment the semaphore (wakes up parent)
    exit(0);
  } else {
    // --- PARENT PROCESS ---
    printf("Parent: Waiting for child to finish.\n");

    sem_down(sem_id); // Value is 0, so parent goes to sleep here!

    printf("Parent: I received the signal. Child is done. Exiting.\n");
    wait(0);
    exit(0);
  }
}
```

Explanation

- **Initialization and Blocking:** The test program `semtest` initializes a global semaphore with a count of 0. After the parent process calls `fork()`, it immediately executes `sem_down()`. Because the count is 0, the parent process enters a high-efficiency blocking state (sleep) using the kernel.
- **Race Condition Evidence:** The scrambled output line (`PareChnt: Waitiilng fod...`) is crucial evidence. It proves that the parent and child processes are truly running concurrently and were rapidly context-switching on the CPU, with their final output characters getting completely scrambled before the parent successfully acquired the semaphore lock and was made to block.
- **Synchronization Proof:** The child process completes its simulated workload (busy loop). Crucially, the final parent output line (`Parent: I received the signal...`) prints only *after* the child process signals completion using `sem_up()`. This successful ordering, and the distinct pause between parental blocking and resumption, proves that

the custom `sleep`, `wakeup`, atomic locking, and IPC-via-semaphore logic are all functioning flawslessly.

2. clone() and join() System Calls (User-Level Threads)
Implemented by: Aashish Kumar

Objective:

The objective of this work is to implement lightweight processes (threads) in the xv6 operating system. Standard xv6 relies entirely on heavyweight processes created via `fork()`, which strictly isolates memory. This implementation introduces the `clone()` and `join()` system calls, allowing the creation of execution threads that share the exact same physical memory (page table) as their parent process, enabling high-performance shared-memory concurrency.

Kernel Modifications

(i) System Call Definition

- **Added `SYS_clone`, `SYS_join` in `syscall.h`.**
- **Declared `sys_clone()` and `sys_join()` in `sysproc.c`.**
- **Added entries in the system call table (`syscall.c`).**
- **Generated user-space stubs via `usys.pl` and exposed prototypes in `user.h`.**

(ii) Process Structure & Memory Management Update

- **Added `is_thread` flag in `struct proc` (`kernel/proc.h`):** Initialized to 0 by default, set to 1 for threads.
- **Implemented `uvmmirror()` in `kernel/vm.c`:** Created a memory mapping function that duplicates a page table structure but points the virtual addresses to the *exact same physical memory pages* as the parent process, avoiding actual memory copying.
- **Updated Garbage Collection in `freeproc()`:** Modified the cleanup logic so that if a terminating process has `is_thread == 1`, the kernel unmaps the page table but explicitly does *not* free the underlying physical RAM, preserving the memory for the parent process.

(iii) System Call Implementation

- **`sys_clone()`:** Extracts the user-defined stack pointer using `argaddr()`. Calls `clone()` which allocates a new process struct, sets `is_thread = 1`, mirrors the parent's memory using `uvmmirror()`, assigns the new stack pointer to the trapframe, and sets the state to `RUNNABLE`.
- **`sys_join()`:** Iterates over the process table to find a `ZOMBIE` child process that specifically possesses the `is_thread == 1` flag. Cleans up the thread resources and returns the thread's PID to the parent.

Core Kernel Implementation (Screenshot Description)

```c
// TMPORTANT: allocproc automatically generated a blank page table.
// Mirror the memory: new page table, but same physical pages!
if(uvmmirror(p->pagetable, np->pagetable, p->sz) < 0){
  freeproc(np);
  release(&np->lock);
  return -1;
}
np->sz = p->sz;

// Copy the parent's register states, but set the Stack Pointer (sp) to the new user stack
*np->trapframe = *p->trapframe;
np->trapframe->a0 = 0;     // Child returns 0
np->trapframe->sp = stack; // The new stack from user space
```

```c
int
uvmmirror(pagetable_t old, pagetable_t new, uint64 sz)
{
  pte_t *pte;
  uint64 pa, i;
  uint flags;

  for(i = 0; i < sz; i += PGSIZE){
    if((pte = walk(old, i, 0)) == 0)
      panic("uvmmirror: pte should exist");
    if((*pte & PTE_V) == 0)
      panic("uvmmirror: page not present");

    pa = PTE2PA(*pte);       // Get the physical address
    flags = PTE_FLAGS(*pte); // Get the permissions

    // Map the NEW pagetable to the OLD physical address!
    if(mappages(new, i, PGSIZE, pa, flags) != 0){
      goto err;
    }
  }
  return 0;
```

```c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

// Global shared variable
int shared_counter = 0;

void thread_function() {
    printf("Thread: Changing the shared counter to 100...\n");
    shared_counter = 100;
    exit(0);
}

int main(int argc, char *argv[]) {
    printf("Parent: Shared counter is initially: %d\n", shared_counter);

    // Allocate 4096 bytes (1 page) for the thread's stack
    void *stack = malloc(4096);

    // Stack grows downwards in memory! So we must pass the TOP of the stack
    int thread_id = clone((void*)((uint64)stack + 4096));

    if(thread_id == 0) {
        // Child thread jumps to the function
        thread_function();
    } else {
        // Parent waits for the thread to finish
        join();
        printf("Parent: Thread joined. Shared counter is now: %d\n", shared_counter);
    }

    exit(0);
}
```

```
$ threadtest
Parent: Shared counter is initially: 0
Thread: Changing the shared counter to 100...
Parent: Thread joined. Shared counter is now: 100
$
```

Explanation

- **Initialization:** The `threadtest` program initializes a global variable (`shared_counter`) to 0.
- **Thread Creation:** The parent allocates 4KB of independent stack memory and calls `clone()`. The kernel successfully creates a thread that maps to the parent's physical memory layout without triggering a trapframe collision.
- **Shared Memory Verification:** The child thread executes `thread_function()`, mutates the global variable to 100, and exits. Standard isolated processes (created via `fork`) cannot alter a parent's global variables.

- **Confirmation:** The parent process successfully waits for the thread's termination using `join()`. The final output confirms the parent reads the counter as 100. This proves the thread successfully operated within the parent's memory space and that true lightweight process concurrency was achieved.

3. procinfo() System Call- Process Information

Implemented by: Aditya

Objective

Implemented a process information subsystem in xv6-riscv that exposes runtime kernel state to user programs via a new system call: **`getpinfo`**. A companion user program **`procinfo`** acts as a process table viewer, printing a formatted snapshot of every active process.

Kernel Data Structures

Two components were added or modified in the kernel to support this feature:

- **`struct pstat`** (new file: `kernel/pstat.h`): A flat, fixed-size snapshot structure that holds per-process data for all `NPROC` process slots in the kernel's process table. Fields include:
  - **`inuse[NPROC]`** — 1 if the slot is occupied, 0 if free.
  - **`pid[NPROC]`** — Process ID of each running process.
  - **`state[NPROC]`** — Numeric encoding of `enum procstate` (UNUSED=0, USED=1, SLEEPING=2, RUNNABLE=3, RUNNING=4, ZOMBIE=5).
  - **`size[NPROC]`** — Virtual memory footprint (`uint64`) of each process in bytes.
  - **`name[NPROC][16]`** — Null-terminated process name string, copied from `struct proc`.
  - **`priority[NPROC]`** — Scheduler priority integer assigned to each process.
- **`struct proc`** (modified: `kernel/proc.h`): The existing PCB was extended with an `int priority` field, which `sys_getpinfo()` reads from each process slot to populate `pstat.priority`.

System Call

- **`sys_getpinfo(struct pstat *st)`** — System call number `SYS_getpinfo (33)`. This is the sole kernel entry point for the feature. Its logic:
  - **Argument extraction:** `argaddr(0, &pstat_addr)` reads the user-space pointer passed by the caller.
  - **Table scan:** Iterates over the global `proc[NPROC]` array. For each slot, it acquires `p->lock` (preventing race conditions), checks `p->state != UNUSED`, and copies PID, state, size, name, and priority into a local `struct pstat` buffer.
  - **Safe copy-out:** `copyout(myproc()->pagetable, pstat_addr, (char*)&st, sizeof(st))` transfers the fully-populated kernel buffer to user-space memory, returning -1 on failure.

User Program — procinfo

The **`procinfo`** command-line utility (`user/procinfo.c`) invokes **`getpinfo()`** and pretty-prints the returned table to the console. It is the primary tool for observing live kernel state from the xv6 shell.

How it works:

- Declares a local `struct pstat st` and calls `getpinfo(&st)`. On failure, it prints an error and exits with code 1.

- Iterates over all `NPROC` slots and, for every slot where `st.inuse[i] == 1`, maps the numeric state to a human-readable label (SLEEPING, RUNNABLE, RUNNING, etc.) and prints a formatted row.

Sample output format:

```
PID      PRIO  STATE        SIZE  NAME
 1 0     SLEEPING     17408 init
 2 0     SLEEPING     28672 sh
 3 0     RUNNING      12288 procinfo
```

Design Decisions & Safety Guarantees

- **Lock discipline:** Each process slot is read only while holding `p->lock`, ensuring the snapshot is internally consistent with no torn reads.

- **Kernel-to-user copy:** The entire buffer is assembled in kernel stack memory first, then transferred with `copyout()`, which validates user-space pointers against the process page table.

· **Read-only:** The system call only reads kernel state and never modifies it, making it inherently safe and side-effect free.

· **No dynamic allocation:** The `struct pstat` is allocated on the kernel stack (fixed size = `NPROC` entries), avoiding heap allocation and fragmentation.

Files Added / Modified

File	Status	Purpose
`kernel/pstat.h`	**NEW**	Defines struct pstat with per-process snapshot fields
`kernel/proc.h`	**MODIFIED**	Added int priority field to struct proc
`kernel/sysproc.c`	**MODIFIED**	Implemented sys_getpinfo() system call handler
`kernel/syscall.h`	**MODIFIED**	Registered SYS_getpinfo (number 33)
`user/procinfo.c`	**NEW**	User-space CLI tool that calls getpinfo() and prints the table
`user/user.h`	**MODIFIED**	Declared getpinfo() prototype
`user/usys.pl`	**MODIFIED**	Added getpinfo syscall stub entry
`Makefile`	**MODIFIED**	Added procinfo to UPROGS build list

4. testsys() System Call

Implemented by**: Enosh**

Objective

The companion program `testsys` exercises all five calls in a single parent-child workflow.

Kernel Data Structures

Three new fields were added to `struct proc` in `kernel/proc.h` to persist per-process IPC and priority state:

- **`int priority`** — Scheduler priority integer. Default value is 0. Can be set and read by the owning process via **`setpriority()`** / **`getpriority()`**.
- **`int msg`** — Holds a single integer message deposited by a sender process.
- **`int has_msg`** — Flag (0 or 1) indicating whether an unread message is waiting. Used by **`sys_recv()`** to spin-wait.

All three fields sit inside the per-process PCB, making them naturally private to each process and always available during context switches.

System Calls

Priority Control (SYS numbers 28 & 29):

- **`sys_getpriority()`** — Returns **`myproc()->priority`** directly as the syscall return value. No arguments. Completely lock-free since the calling process is reading only its own field.
- **`sys_setpriority(int p)`** — Extracts integer argument via **`argint(0, &p)`** and writes it into **`myproc()->priority`**. Returns 0 on success.

IPC — Message Passing (SYS numbers 30 & 31):

- **`sys_send(int pid, int msg)`** — Scans the entire `proc[]` table to find the process with matching PID. Acquires `p->lock` for each entry, writes `msg` into the target's `p->msg`, sets `p->has_msg = 1`, then releases the lock. Returns -1 if no matching PID is found.
- **`sys_recv()`** — Busy-spins on `myproc()->has_msg == 0` until a message arrives. Once a message is present it copies `p->msg` into a local variable, clears `has_msg`, and returns the integer value to the caller.

Signal (SYS number 32):

- **`sys_signal(int pid)`** — A thin wrapper around the kernel's existing `kkill(pid)` function. Delivers a termination signal to the target process identified by PID.

User Program — testsys

The `testsys` program (`user/testsys.c`) is an end-to-end integration test that exercises all five system calls in a single execution run:

- Priority setting/reading: Calls `setpriority(5)` then immediately reads it back with `getpriority()` and prints the result. Confirms the value round-trips correctly.
- Message passing (parent → child): `fork()` creates a child. The parent calls `send(pid, 42)` to deposit the integer 42 into the child's message slot. The child calls `recv()` and prints the received value.
- Signal: After the IPC exchange, `signal(getpid())` is called, sending a kill signal to the calling process itself as a self-signal demonstration.

Expected output:

```
Priority: 5
 Child got: 42
```

Design Notes & Limitations

- Spin-wait in recv(): `sys_recv()` uses a busy-wait loop, which consumes CPU while waiting. A production system would call sleep() to yield the CPU; this is noted as a potential improvement.
- Single-message buffer: Each process can hold only one pending message at a time. A second `send()` before `recv()` is called will silently overwrite the first message.
- Lock discipline in send(): The table scan acquires and releases each process lock individually, ensuring no other CPU can concurrently modify the target process's state during the write.
- signal() wraps kkill(): Rather than duplicating termination logic, `sys_signal()` delegates to the already-tested `kkill()` kernel function, keeping the implementation minimal.

Files Added / Modified

File	Status	Purpose
`kernel/proc.h`	**MODIFIED**	Added int priority, int msg, int has_msg to struct proc

`kernel/sysproc.c`	**MODIFIED**	Implemented sys_getpriority, sys_setpriority, sys_send, sys_recv, sys_signal
`kernel/syscall.h`	**MODIFIED**	Registered SYS_getpriority(28), SYS_setpriority(29), SYS_send(30), SYS_recv(31), SYS_signal(32)
`kernel/syscall.c`	**MODIFIED**	Added extern declarations and dispatch table entries for all 5 syscalls
`user/testsys.c`	**NEW**	End-to-end integration test for all five system calls
`user/user.h`	**MODIFIED**	Declared getpriority, setpriority, send, recv, signal prototypes
`user/usys.pl`	**MODIFIED**	Added stub entries for all 5 syscalls
`Makefile`	**MODIFIED**	Added testsys to UPROGS build list

5. myfork() System Call *by Aditya Ekka*

a custom system call that limits child process creation to a count for a given parent.

myfork.c | sysproc.c | proc.h | Makefile M | syscall.h | syscall.c | user.h | usys.pl

```makefile

UPROGS=\
	$U/_cat\
	$U/_echo\
	$U/_forktest\
	$U/_grep\
	$U/_init\
	$U/_kill\
	$U/_ln\
	$U/_ls\
	$U/_mkdir\
	$U/_rm\
	$U/_sh\
	$U/_stressfs\
	$U/_usertests\
	$U/_grind\
	$U/_wc\
	$U/_zombie\
	$U/_logstress\
	$U/_forphan\
	$U/_dorphan\
	$U/_ps\
	$U/_semtest\
	$U/_threadtest\
	$U/_myfork\

fs.img: mkfs/mkfs README $(UPROGS)
	mkfs/mkfs fs.img README $(UPROGS)

-include kernel/*.d user/*.d

clean:
	rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
	*/*.o */*.d */*.asm */*.sym \
	$K/kernel fs.img \
```

myfork.c | sysproc.c | proc.h | Makefile M | syscall.h M | syscall.c M | user.h | usys.pl

```c
extern uint64 sys_clone(void);
extern uint64 sys_join(void);
extern uint64 sys_myfork(void); //

// An array mapping syscall numbers from syscall.h
// to the function that handles the system call.
static uint64 (*syscalls[])(void) = {
[SYS_fork]    sys_fork,
[SYS_exit]    sys_exit,
[SYS_wait]    sys_wait,
[SYS_pipe]    sys_pipe,
[SYS_read]    sys_read,
[SYS_kill]    sys_kill,
[SYS_exec]    sys_exec,
[SYS_fstat]   sys_fstat,
[SYS_chdir]   sys_chdir,
[SYS_dup]     sys_dup,
[SYS_getpid]  sys_getpid,
[SYS_sbrk]    sys_sbrk,
[SYS_pause]   sys_pause,
[SYS_uptime]  sys_uptime,
[SYS_open]    sys_open,
[SYS_write]   sys_write,
[SYS_mknod]   sys_mknod,
[SYS_unlink]  sys_unlink,
[SYS_link]    sys_link,
[SYS_mkdir]   sys_mkdir,
[SYS_close]   sys_close,
[SYS_getprocs] sys_getprocs,
[SYS_sem_create] sys_sem_create,
[SYS_sem_down]   sys_sem_down,
[SYS_sem_up]     sys_sem_up,
[SYS_clone] sys_clone,
[SYS_join]  sys_join,
[SYS_myfork] sys_myfork, //
};
```

myfork.c | sysproc.c | proc.h | Makefile M | syscall.h M | syscall.c M | user.h M | usys.pl

```c

struct stat;

// system calls
int fork(void);
int exit(int) __attribute__((noreturn));
int wait(int*);
int pipe(int*);
int write(int, const void*, int);
int read(int, void*, int);
int close(int);
int kill(int);
int exec(const char*, char**);
int open(const char*, int);
int mknod(const char*, short, short);
int unlink(const char*);
int fstat(int fd, struct stat*);
int link(const char*, const char*);
int mkdir(const char*);
int chdir(const char*);
int dup(int);
int getpid(void);
char* sys_sbrk(int,int);
int pause(int);
int uptime(void);
int getprocs(void);
int sem_create(int);
int sem_down(int);
int sem_up(int);
int clone(void*);
int join(void);
int myfork(void); //
// ulib.c
int stat(const char*, struct stat*);
char* strcpy(char*, const char*);
void *memmove(void*, const void*, int);
```

myfork.c | sysproc.c | proc.h | Makefile M | syscall.h M | syscall.c M | user.h M | usys.pl M

```perl
sub entry {
    print " li a7, SYS_${name}\n";
    print " ecall\n";
    print " ret\n";
}

entry("fork");
entry("exit");
entry("wait");
entry("pipe");
entry("read");
entry("write");
entry("close");
entry("kill");
entry("exec");
entry("open");
entry("mknod");
entry("unlink");
entry("fstat");
entry("link");
entry("mkdir");
entry("chdir");
entry("dup");
entry("getpid");
entry("sbrk");
entry("pause");
entry("uptime");
entry("getprocs");
entry("sem_create");
entry("sem_down");
entry("sem_up");
entry("clone");
entry("join");
entry("myfork"); //
```

myfork.c | sysproc.c M | proc.h | Makefile M | syscall.h M | syscall.c M | user.h M | usys.pl M

```c
// System call numbers
#define SYS_fork    1
#define SYS_exit    2
#define SYS_wait    3
#define SYS_pipe    4
#define SYS_read    5
#define SYS_kill    6
#define SYS_exec    7
#define SYS_fstat   8
#define SYS_chdir   9
#define SYS_dup    10
#define SYS_getpid 11
#define SYS_sbrk   12
#define SYS_pause  13
#define SYS_uptime 14
#define SYS_open   15
#define SYS_write  16
#define SYS_mknod  17
#define SYS_unlink 18
#define SYS_link   19
#define SYS_mkdir  20
#define SYS_close  21
#define SYS_getprocs 22
#define SYS_sem_create 23
#define SYS_sem_down   24
#define SYS_sem_up     25
#define SYS_clone 26
#define SYS_join 27
#define SYS_myfork 28
```

myfork.c | sysproc.c M | proc.h M | Makefile M | syscall.h M | syscall.c M | user.h M | usys.pl M

```c

uint64 sys_myfork(void) {
  int count = 0;
  struct proc *p;
  struct proc *cur = myproc();
  const int max_children = 5;

  for(p = proc; p < &proc[NPROC]; p++) {
    if(p->parent == cur)
      count++;
  }

  if(count >= max_children) {
    return -1;
  }

  return kfork();
}
```

myfork.c | myfork.as | sysproc.c | proc.h M | Makefile | syscall.h | syscall.c | user.h M | usys.pl

```c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int main()
{
  int pid;
  int i;

  for(i = 1; i <= 6; i++) {
    pid = myfork();
    if(pid < 0) break;
    if(pid == 0) exit(0);
    printf("parent %d created child %d pid=%d\n", getpid(), i, pid);
  }
  while(wait(0) >= 0);

  exit(0);
}

//test file to check functionality of the custom myfork system call, which limits the number of child processes that can be created by a parent process to 5. The program attempts to create 6 child processes, and the expected output is that the first 5 child processes are created successfully, while the 6th attempt should fail with a message indicating that the maximum number of child processes has been reached.
//written by Aditya Ekka
```